

Experiment 6

Student Name: Ashutosh Yadav
Branch: CSE
Semester: 6th
Subject Name: System Design

UID: 23BCS11023
Section/Group: KRG 3-B
Date of Performance: 25/02/2026
Subject Code: 23CSH-314

Aim: To design and analyze a scalable video-sharing and streaming platform similar to YouTube that supports video uploads, adaptive streaming (HLS/DASH), likes, comments, and massive concurrent users with high availability and low latency.

Objective:

- To identify functional and non-functional requirements of a YouTube-like system
- To design High-Level Architecture (HLD) for a video-sharing platform
- To understand Manifest files and Adaptive Bitrate Streaming
- To implement Like and Comment handling in distributed systems
- To understand the role of CDN, Kafka, Cache, and Object Storage
- To apply scalability and CAP theorem concepts

Tools:

- **Draw.io** – System Architecture & HLD Diagram
- **MySQL** – User, subscription, and payment data
- **MongoDB / NoSQL DB** – Video metadata storage
- **ElasticSearch** – Search functionality for movies and TV shows
- **Apache Kafka** – Asynchronous event streaming and video pipeline coordination
- **Redis / CDN Cache** – Caching frequently accessed content • **Blob Storage (Amazon S3 equivalent)** – Video and media storage

System Requirements:

A. Functional Requirements:-

1. Client should be able to upload videos on the platform.
2. Client should be able to search for videos based on the video title or names.
3. Client should be able to watch videos in multiple different resolutions
 - 480p
 - 720p
 - 1080p
 - 4K etc.

4. User should be able to like videos
5. User should be able to view and post comments on videos

B. Non-Functional Requirements:-

A. Scalability

System must handle:

- 2B+ users
- Millions of concurrent streams
- 500+ hours uploaded per minute

Must Support:

- Horizontal scaling
- Auto-scaling
- Sharding
- CDN edge caching

B. Performance

Latency Requirements

Feature	Latency Target
Video start	< 2 sec
Search	< 200 ms
Like	< 100 ms
Comment load	< 300 ms

High Level Design (HLD):

USER UPLOADING A VIDEO (Detailed Flow)

Step 1 — User Selects Video

User chooses a file in the client app.

Client reads:

- file size
- resolution
- codec

- duration

Client sends request:

POST /videos/initiate-upload

Step 2 — Authentication & Upload Session

API Gateway:

- validates token
- rate limits request

Upload Service:

- generates **video_id**
- creates metadata entry:

```
video_id
  user_id
  status = UPLOADING
  visibility = PRIVATE
  upload_time
```

Returns:

```
video_id
  chunk_size
  signed_upload_urls
```

Step 3 — Chunked Upload to Object Storage

Instead of sending to app server:

Client → CDN → Object Storage

Client uploads chunks (5–10MB each).

Benefits:

- resumable uploads
 - reduces backend load
 - supports large files (GBs)
-

Step 4 — Upload Completion

Client notifies backend:

POST /videos/{video_id}/complete

Upload Service:

- verifies all chunks

- marks status:

PROCESSING

Pushes job to **video processing queue**.

Step 5 — Transcoding Pipeline

Worker retrieves raw video from storage.

Video converted into multiple bitrates:

Resolution	Bitrate
144p	200 kbps
360p	800 kbps
720p	2.5 Mbps
1080p	5 Mbps

Step 6 — Segment Generation

Video is split into **small segments** (2–4 seconds).

Two formats generated:

HLS

.ts segments
.m3u8 playlist

DASH

.m4s segments
.mpd manifest

Step 7 — Manifest File Creation

HLS master manifest example

```
#EXTM3U
#EXT-X-STREAM-INF:BANDWIDTH=800000,RESOLUTION=640x360
360p/playlist.m3u8

#EXT-X-STREAM-INF:BANDWIDTH=2500000,RESOLUTION=1280x720
720p/playlist.m3u8
```

DASH manifest

```
<AdaptationSet>
  <Representation id="360p" bandwidth="800000"/>
  <Representation id="720p" bandwidth="2500000"/>
</AdaptationSet>
```

Manifest tells player:

- available bitrates
- segment URLs
- codecs

Step 8 — Thumbnail Generation

Thumbnail service extracts frames:

0:10
0:30
1:00

Generates multiple sizes:

120x90
320x180
1280x720

Stored in object storage.

Step 9 — Thumbnail Proxy Service

Instead of direct storage access:

Client → CDN → Thumbnail Proxy → Storage

Benefits:

- resizing
- caching
- security
- compression

Step 10 — Metadata Update

Metadata DB updated:

status = READY
duration
available_qualities
thumbnail_urls
manifest_url

Search index updated.

USER SEARCHING FOR A VIDEO

Step 1 — User enters query

Example:

"system design tutorial"

Client sends:

GET /search?q=system+design+tutorial

Step 2 — Query preprocessing

Search service:

- tokenization
- stemming
- stop-word removal

Example:

system design tutorial

becomes

system
design
tutorial

Step 3 — Inverted Index Search

Search index stores:

word → video_ids

Example:

system → [v1, v5, v12]
design → [v2, v5]
tutorial → [v3, v5]

Intersection → results.

Step 4 — Search Optimization

Techniques used:

Caching

Popular queries cached.

Query suggestions

"system design interview"

Ranking factors

- relevance
 - watch time
 - click-through rate
 - engagement
 - personalization
-

Step 5 — Fetch metadata

Search returns video_ids.

Metadata service fetches:

title
channel
duration
thumbnail_url
views

From:

- Redis cache
 - Metadata DB
-

Step 6 — Thumbnail Proxy Delivery

Results include thumbnail URLs.

Images served through:

Client → CDN → Thumbnail Proxy → Storage

This avoids direct storage exposure.

USER WATCHING A VIDEO

Step 1 — Video page request

User clicks video.

Client requests:

GET /videos/{video_id}

Metadata returned:

manifest_url
title
views
duration

Step 2 — Player requests manifest

Example:

GET /videos/abc/master.m3u8

CDN serves manifest.

Step 3 — Manifest analysis

Player reads available streams:

360p
720p
1080p

Step 4 — Bandwidth estimation

Player downloads first segment.

Example:

segment_size = 2MB
download_time = 1 sec
bandwidth $\approx$ 16 Mbps

Player chooses safe bitrate.

Step 5 — Segment fetching

Video segments fetched:

Client → CDN → Storage

Example:

segment_1.ts
segment_2.ts
segment_3.ts

Each = 2–4 seconds.

Step 6 — Adaptive bitrate switching

If bandwidth drops:

5 Mbps → 1 Mbps

Player switches:

720p → 360p

Manifest allows this dynamic switching.

Step 7 — Buffer management

Player maintains buffer:

15–30 seconds

If buffer low:

- downgrade quality.

If buffer high:

- upgrade quality.
-

Step 8 — CDN caching

Popular segments cached at edge.

Flow:

Client → CDN edge → segment

Only cache miss goes to storage.

Thumbnail Proxy Architecture

Purpose:

- resizing
- format conversion
- caching
- protection

Flow:

Client



CDN



Thumbnail Proxy



Object Storage

Proxy may:

- convert JPG → WebP
- generate responsive sizes

Handling Likes

Requirements

- Prevent duplicate likes
- Very high write throughput
- Very fast read for like count
- No pub-sub system

Database Design

We maintain two things:

Like Mapping Table

Stores which user liked which video.

likes

user_id

video_id

created_at

PRIMARY KEY (user_id, video_id)

This ensures:

- one like per user per video

Video Metadata Table

videos

video_id
title
like_count
comment_count
status

Like Flow (Step-by-Step)

Step 1 — User Clicks Like

Client sends request

POST /videos/{video_id}/like

Step 2 — API Gateway

API authenticates user and forwards request to Like Service.

Step 3 — Check Existing Like

Like Service checks in DB or cache:

```
SELECT * FROM likes  
  WHERE user_id=? AND video_id=?
```

Step 4 — Insert Like

If no record exists:

```
INSERT INTO likes(user_id, video_id)
```

Step 5 — Update Counter

Instead of writing to DB each time:

Update counter in Redis

```
INCR video:{video_id}:likes
```

This handles millions of writes efficiently.

Step 6 — Periodic DB Sync

A background worker periodically updates the main DB.

```
UPDATE videos
```

```
SET like_count = redis_value  
WHERE video_id = ?
```

This avoids heavy write load.

Step 7 — Response

Server returns updated like count.

Like Read Flow

When a video page loads:

Client → API → Redis → Like Count

If cache miss:

Redis → Metadata DB

Handling Comments

Requirements

- Ordered comments
- Pagination
- Replies
- Moderate write load
- No pub-sub

Database Schema

comments

```
-----  
comment_id  
video_id  
user_id  
parent_comment_id  
text  
created_at
```

Index:

(video_id, created_at DESC)

Comment Write Flow

Step 1 — User Posts Comment

POST /videos/{video_id}/comments

Step 2 — Comment Service

Service writes directly to DB.

INSERT INTO comments(...)

Step 3 — Update Comment Count

Update Redis counter.

INCR video:{video_id}:comment_count

Step 4 — Response

Return newly created comment.

Comment Read Flow

Client requests comments.

GET /videos/{video_id}/comments?cursor=123

Server queries:

```
SELECT *  
  FROM comments  
 WHERE video_id=?  
 ORDER BY created_at DESC  
  LIMIT 20
```

Cursor-based pagination used.

Real-Time Comment Updates (No Pub-Sub)

Instead of pub-sub we use polling.

Client periodically calls:

GET /videos/{video_id}/comments?after_timestamp=xxx

Server returns only new comments.



7. Learning Outcomes :

- Understand how large-scale video platforms manage **video uploads, processing, and storage**.
- Learn how **HLS and DASH adaptive streaming** works using manifest files.
- Understand the architecture behind **efficient video search using inverted indexes and ranking**.
- Learn how **likes and comments are handled at scale** using caching and distributed databases.
- Understand the role of **CDNs and thumbnail proxy services** in reducing latency and bandwidth usage.
- Gain insight into designing **scalable, fault-tolerant distributed systems** for billions of users.